SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture. *(BL3)*

Assessment Tool Weightage: 60%

QUESTIONS: 6 **Total Marks:60**

Annexure A

Application Based Questions

Code of Conduct:

I Dhaanush Rajan(192421162) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Application Based Questions

1. Use Case Diagram Analysis

A **Smart Hotel Reservation System** allows guests to search rooms, make reservations, cancel bookings, and process payments. Managers handle room allocation and reports, while payment services process transactions. The current design does not clearly distinguish actor responsibilities.

Analyze the system requirements, identify missing actors and use cases, and design an optimized Use Case Diagram showing proper relationships (include, extend, generalization) for better system clarity.

(10 Marks)

ANSWER

1. System Requirements Analysis

The current system supports:

- Guests search for available rooms.
- Guests make reservations.
- Guests cancel reservations.
- Guests make payments.
- Managers allocate rooms.
- Managers generate reports.
- Payment Service processes online transactions.

Missing Actors

To improve the system, the following actors should be added:

1. **Receptionist** – Handles guest check-in, check-out, and booking modifications.
2. **Admin** – Manages users, rooms, pricing, and hotel settings.
3. **Notification Service** – Sends booking confirmations, cancellation alerts, and payment receipts.

2. Actors

Actor	Responsibilities
Guest	Search rooms, reserve rooms, cancel booking, make payment, receive confirmation
Receptionist	Check-in guest, Check-out guest, Modify reservation

Actor	Responsibilities
Manager	Allocate rooms, Generate reports
Admin	Manage rooms, Manage users, Update room prices
Payment Service	Process online payments
Notification Service	Send booking and payment notifications

---

3. Use Cases

Guest

- Register/Login
- Search Rooms
- View Room Details
- Check Availability
- Make Reservation
- Modify Reservation
- Cancel Reservation
- Make Payment
- View Booking History

Receptionist

- Check-in Guest
- Check-out Guest
- Update Reservation

Manager

- Allocate Room
- Generate Reports
- View Occupancy Status

Admin

- Manage Rooms
- Manage Users
- Update Pricing
- Manage Discounts

Payment Service

- Verify Payment
- Process Transaction

Notification Service

- Send Booking Confirmation
- Send Cancellation Notification
- Send Payment Receipt

4. Relationships

<<include>>

These are mandatory sub-functions.

- **Make Reservation** <<include>> **Check Availability**
- **Make Reservation** <<include>> **Process Payment**
- **Process Payment** <<include>> **Verify Payment**
- **Cancel Reservation** <<include>> **Refund Payment** (if applicable)
- **Make Reservation** <<include>> **Send Booking Confirmation**
- **Cancel Reservation** <<include>> **Send Cancellation Notification**

<<extend>>

Optional or conditional behavior.

- **Modify Reservation** <<extend>> **Make Reservation**
- **View Room Details** <<extend>> **Search Rooms**
- **Apply Discount** <<extend>> **Process Payment**
- **Generate Occupancy Report** <<extend>> **Generate Reports**

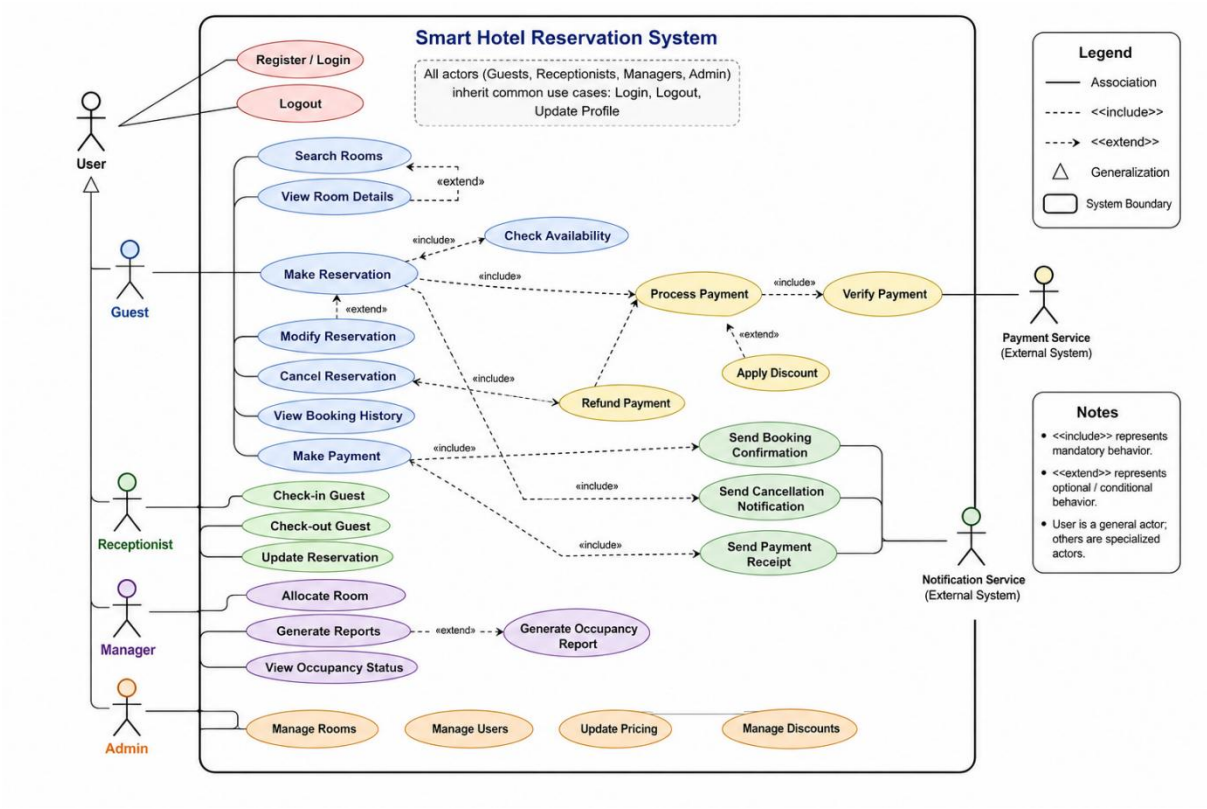
Generalization

User

- Guest
- Receptionist
- Manager
- Admin

All inherit common actions such as:

- Login
- Logout
- Update Profile


Smart Hotel Reservation System
All actors (Guests, Receptionists, Managers, Admin) inherit common use cases: Login, Logout, Update Profile
Register / Login
Logout
Search Rooms
View Room Details
Check Availability
Make Reservation
Process Payment
Verify Payment
Modify Reservation
Cancel Reservation
View Booking History
Make Payment
Refund Payment
Apply Discount
Send Booking Confirmation
Send Cancellation Notification
Send Payment Receipt
Check-in Guest
Check-out Guest
Update Reservation
Allocate Room
Generate Reports
Generate Occupancy Report
View Occupancy Status
Manage Rooms
Manage Users
Update Pricing
Manage Discounts
«extend»
«include»
User
Guest
Receptionist
Manager
Admin
Payment Service (External System)
Notification Service (External System)
Legend
Association
<<include>>
<<extend>>
Generalization
System Boundary
Notes
<<include>> represents mandatory behavior.
<<extend>> represents optional / conditional behavior.
User is a general actor; others are specialized actors.

2. Class Diagram Design

A **University Management System** includes entities such as Student, Faculty, Course, Department, and Examination. The existing design lacks proper attributes, operations, and relationships among classes.

Analyze the system, identify relevant classes with attributes and methods, define relationships (association, aggregation, inheritance), and construct a well-structured Class Diagram.

(10 Marks)

ANSWER

1. System Analysis

The University Management System manages students, faculty, departments, courses, and examinations. The existing design lacks:

- Proper class attributes
- Member functions (operations)
- Relationships between classes
- Inheritance among common entities

2. Classes with Attributes and Operations

1. Person (Super Class)

Attributes

- personID : int
- name : String
- email : String
- phone : String

Methods

- login()
- logout()
- updateProfile()

2. Student (inherits Person)

Attributes

- studentID : int
- semester : int
- CGPA : float

Methods

- enrollCourse()
- viewResult()
- payFees()

3. Faculty (inherits Person)

Attributes

- facultyID : int
- designation : String
- specialization : String

Methods

- assignMarks()
- teachCourse()
- generateAttendance()

4. Department

Attributes

- deptID : int
- deptName : String
- location : String

Methods

- addFaculty()
- addStudent()
- offerCourse()

5. Course

Attributes

- courseID : int
- courseName : String
- credits : int

Methods

- assignFaculty()
- enrollStudent()
- displayCourse()

6. Examination

Attributes

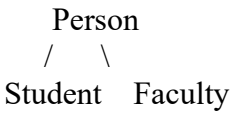
- examID : int
- examType : String
- examDate : Date
- totalMarks : int

Methods

- scheduleExam()
- publishResult()
- calculateGrade()

3. Relationships

Generalization (Inheritance)

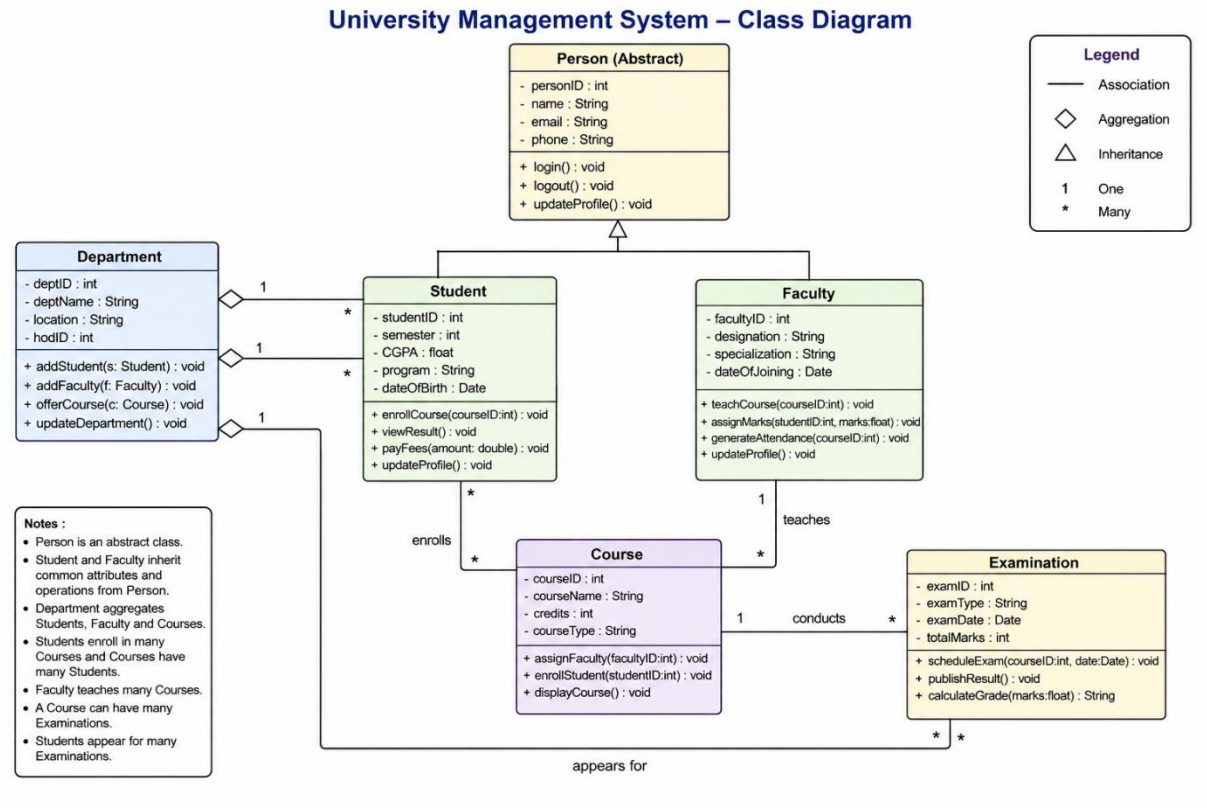


Association

- Student enrolls in Course
- Faculty teaches Course
- Student appears for Examination

Aggregation

- Department contains Students
- Department contains Faculty
- Department offers Courses


University Management System – Class Diagram
Person (Abstract)
- personID : int
- name : String
- email : String
- phone : String
+ login() : void
+ logout() : void
+ updateProfile() : void
Legend
Association
Aggregation
Inheritance
1 One
* Many
Department
- deptID : int
- deptName : String
- location : String
- hodID : int
+ addStudent(s: Student) : void
+ addFaculty(f: Faculty) : void
+ offerCourse(c: Course) : void
+ updateDepartment() : void
Student
- studentID : int
- semester : int
- CGPA : float
- program : String
- dateOfBirth : Date
+ enrollCourse(courseID:int) : void
+ viewResult() : void
+ payFees(amount: double) : void
+ updateProfile() : void
Faculty
- facultyID : int
- designation : String
- specialization : String
- dateOfJoining : Date
+ teachCourse(courseID:int) : void
+ assignMarks(studentID:int, marks:float) : void
+ generateAttendance(courseID:int) : void
+ updateProfile() : void
Course
- courseID : int
- courseName : String
- credits : int
- courseType : String
+ assignFaculty(facultyID:int) : void
+ enrollStudent(studentID:int) : void
+ displayCourse() : void
Examination
- examID : int
- examType : String
- examDate : Date
- totalMarks : int
+ scheduleExam(courseID:int, date:Date) : void
+ publishResult() : void
+ calculateGrade(marks:float) : String
enrolls
teaches
conducts
appears for
Notes :
• Person is an abstract class.
• Student and Faculty inherit common attributes and operations from Person.
• Department aggregates Students, Faculty and Courses.
• Students enroll in many Courses and Courses have many Students.
• Faculty teaches many Courses.
• A Course can have many Examinations.
• Students appear for many Examinations.

3. Sequence Diagram Evaluation

In a **Vehicle Rental System**, a customer searches for a vehicle, books it, makes payment, and receives a rental confirmation. The interaction among system components is incomplete and poorly documented.

Analyze the process flow, identify missing objects and message sequences, and design a detailed Sequence Diagram representing proper interaction among system components.

(10 Marks)

ANSWER

Sequence Diagram Explanation – Vehicle Rental System (10 Marks)

A **Sequence Diagram** illustrates how objects interact with one another over time to complete the vehicle rental process. Time flows from **top to bottom**, and each vertical line (lifeline) represents an object participating in the interaction.

Objects (Lifelines)

1. **Customer** – Searches for and books a vehicle.
2. **Web/Mobile Application** – User interface through which the customer interacts.
3. **Vehicle Rental System** – Controls the entire booking process.
4. **Vehicle Database** – Stores vehicle availability and details.
5. **Booking Service** – Creates and manages reservations.
6. **Payment Gateway** – Processes online payments.
7. **Notification Service** – Sends booking confirmation to the customer.

Step-by-Step Explanation

Step 1: Search Vehicle

- The **Customer** enters search criteria (vehicle type, pickup date, return date) in the **Web/Mobile Application**.
- The application forwards the request to the **Vehicle Rental System**.
- The system queries the **Vehicle Database** to check available vehicles.
- The database returns the list of available vehicles.
- The system displays the available vehicles to the customer.

Purpose: To allow the customer to view only available vehicles.

Step 2: Select Vehicle and Create Booking

- The customer selects a vehicle and submits booking details.
- The application sends the booking request to the **Vehicle Rental System**.
- The system calls the **Booking Service**.
- The **Booking Service** reserves the selected vehicle by updating the **Vehicle Database**.

- The database confirms that the vehicle has been reserved.
- The booking service returns the booking details (Booking ID, amount, vehicle information).
- The booking summary is displayed to the customer.

Purpose: To reserve the vehicle before payment.

Step 3: Make Payment

- The customer enters payment details.
- The application sends the payment request to the **Vehicle Rental System**.
- The system forwards the request to the **Payment Gateway**.
- The payment gateway verifies and processes the payment.
- If successful, it returns a **Payment Success** message along with a transaction ID.

Purpose: To securely complete the payment.

Step 4: Confirm Booking

- After receiving successful payment confirmation, the **Vehicle Rental System** updates the booking status to **Confirmed**.
- The **Booking Service** updates the reservation status in the database.
- The system receives confirmation that the booking has been successfully updated.

Purpose: To ensure bookings are confirmed only after successful payment.

Step 5: Send Rental Confirmation

- The **Vehicle Rental System** sends the booking details to the **Notification Service**.
- The notification service sends the rental confirmation (Email, SMS, or Mobile App Notification) to the customer.
- The customer receives the booking confirmation.

Purpose: To notify the customer that the booking has been successfully completed.

UML Symbols Used

UML Symbol	Meaning
Actor (Customer)	External user interacting with the system
Lifeline (Vertical dashed line)	Represents the existence of an object during the interaction
Activation Bar	Indicates the period when an object is performing an operation
Solid Arrow	Synchronous method call (request)
Dashed Arrow	Return message or response

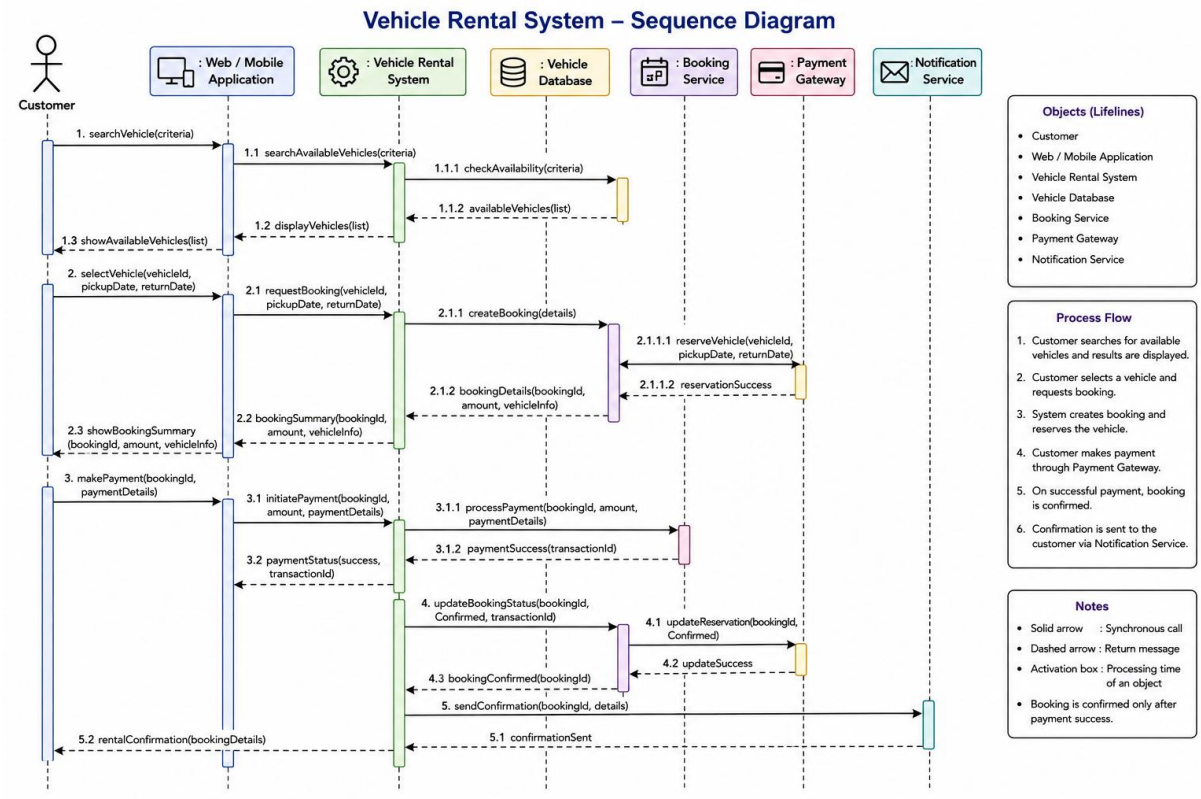
UML Symbol	Meaning
Time Direction	Flows from top to bottom

Advantages of the Optimized Sequence Diagram

- Clearly defines the responsibilities of each component.
- Separates user interface, business logic, database, payment, and notification services.
- Ensures the vehicle is reserved before payment.
- Confirms the booking only after successful payment.
- Sends automatic confirmation notifications to the customer.
- Improves system modularity, scalability, and maintainability.

Conclusion

The optimized sequence diagram accurately models the **end-to-end vehicle rental process**. It starts with the customer searching for a vehicle, proceeds through availability checking, booking, payment processing, booking confirmation, and ends with a rental confirmation notification. The diagram follows UML best practices by showing the correct sequence of interactions, message flow, activation periods, and responsibilities of all system components. This design is more complete, organized, and suitable for implementation in a real-world Vehicle Rental System.



4. State Diagram Construction

A **Traffic Signal Control System** operates through states such as Red Signal, Green Signal, Yellow Signal, Emergency Mode, and Maintenance Mode. The transitions between states are not clearly defined.

Analyze the system behavior, identify valid states and transitions, and construct a State Transition Diagram ensuring all events and conditions are properly represented.

(10 Marks)

ANSWER

1. System Analysis

A **Traffic Signal Control System** controls the flow of vehicles by changing traffic lights through different operational states. The existing system does not clearly define the transitions between states.

The system normally cycles through:

- Red Signal
- Green Signal
- Yellow Signal

Additionally, it supports:

- Emergency Mode (for ambulances, fire engines, police)
- Maintenance Mode (during repair or testing)

2. Valid States

State	Description
Initial State	System starts
Red Signal	Vehicles must stop
Green Signal	Vehicles are allowed to move
Yellow Signal	Warns that the signal is about to change
Emergency Mode	Gives priority to emergency vehicles
Maintenance Mode	Signal is under maintenance or flashing mode
Final State	System shutdown

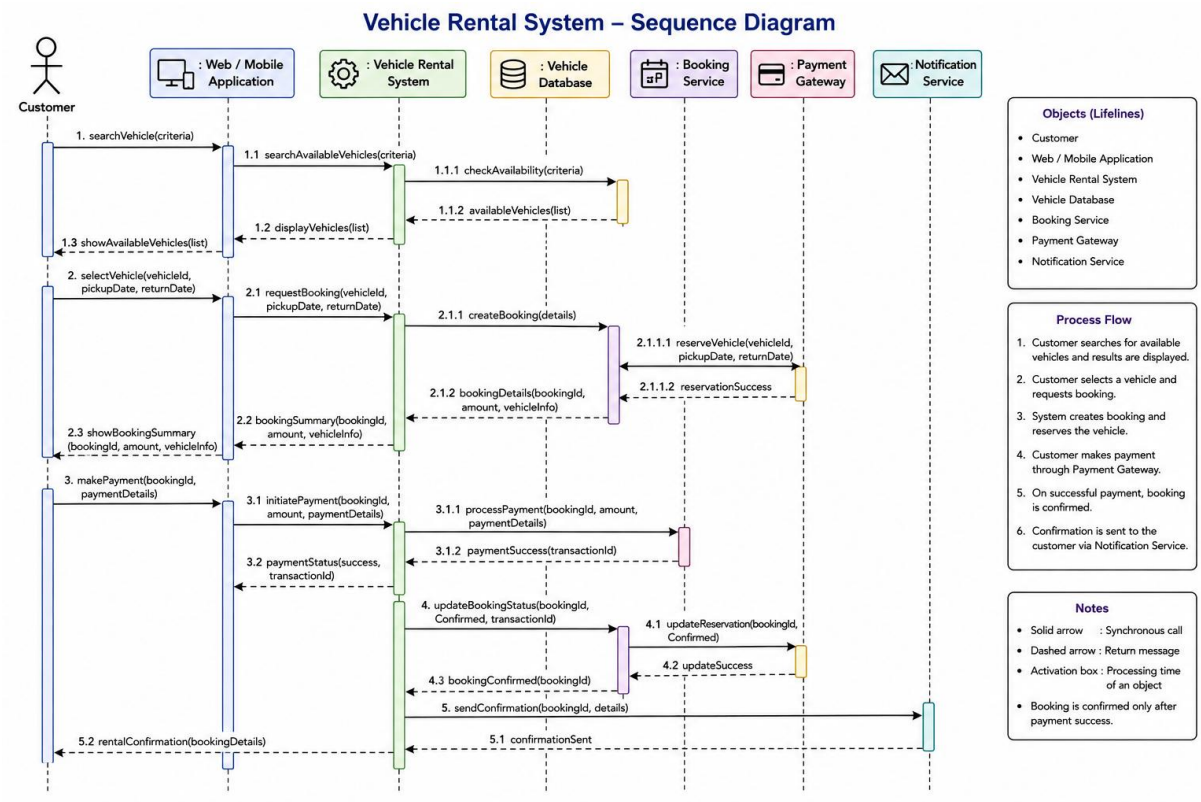
3. Events Causing State Transitions

Current State	Event	Next State
Initial	System Start	Red Signal
Red Signal	Timer Expires	Green Signal

Current State	Event	Next State
Green Signal	Timer Expires	Yellow Signal
Yellow Signal	Timer Expires	Red Signal
Any Normal State	Emergency Vehicle Detected	Emergency Mode
Emergency Mode	Emergency Cleared	Red Signal
Any State	Maintenance Requested	Maintenance Mode
Maintenance Mode	Maintenance Completed	Red Signal
Red Signal	System Shutdown	Final State

Improvements Over Existing Design

- Clearly defines all operational states.
- Specifies valid transitions between states.
- Includes exceptional states such as **Emergency Mode** and **Maintenance Mode**.
- Uses event-driven transitions for better control.
- Improves safety, readability, and maintainability.
- Follows UML State Diagram standards.



5. Component Diagram Design

A **Smart Healthcare Portal** consists of modules such as Patient Management, Appointment Scheduling, Medical Records, Billing, and Notification Services. The current architecture does not clearly define component interactions and dependencies.

Analyze the system architecture, identify major components and their interactions, and design a Component Diagram showing dependencies and interfaces between modules.

(10 Marks)

ANSWER

1. System Analysis

A **Smart Healthcare Portal** integrates multiple software components to provide healthcare services. The existing architecture does not clearly define how different modules communicate and depend on each other.

The major components are:

- Patient Management
- Appointment Scheduling
- Medical Records
- Billing
- Notification Service
- Database (Shared Repository)

2. Major Components

Component	Responsibility
Patient Management	Registers patients, updates profiles, manages patient information
Appointment Scheduling	Books, updates, and cancels appointments
Medical Records	Stores diagnosis, prescriptions, lab reports, and treatment history
Billing	Generates invoices, processes payments, and maintains billing history
Notification Service	Sends SMS/Email notifications for appointments, payments, and reports
Healthcare Database	Stores all patient, appointment, billing, and medical record data

---

3. Component Interactions

- **Patient Management** interacts with **Appointment Scheduling** for booking appointments.

- **Appointment Scheduling** accesses **Medical Records** before confirming appointments.
- **Billing** retrieves patient details from **Patient Management** and appointment details from **Appointment Scheduling**.
- **Notification Service** receives updates from all modules and sends notifications.
- All components communicate with the **Healthcare Database**.

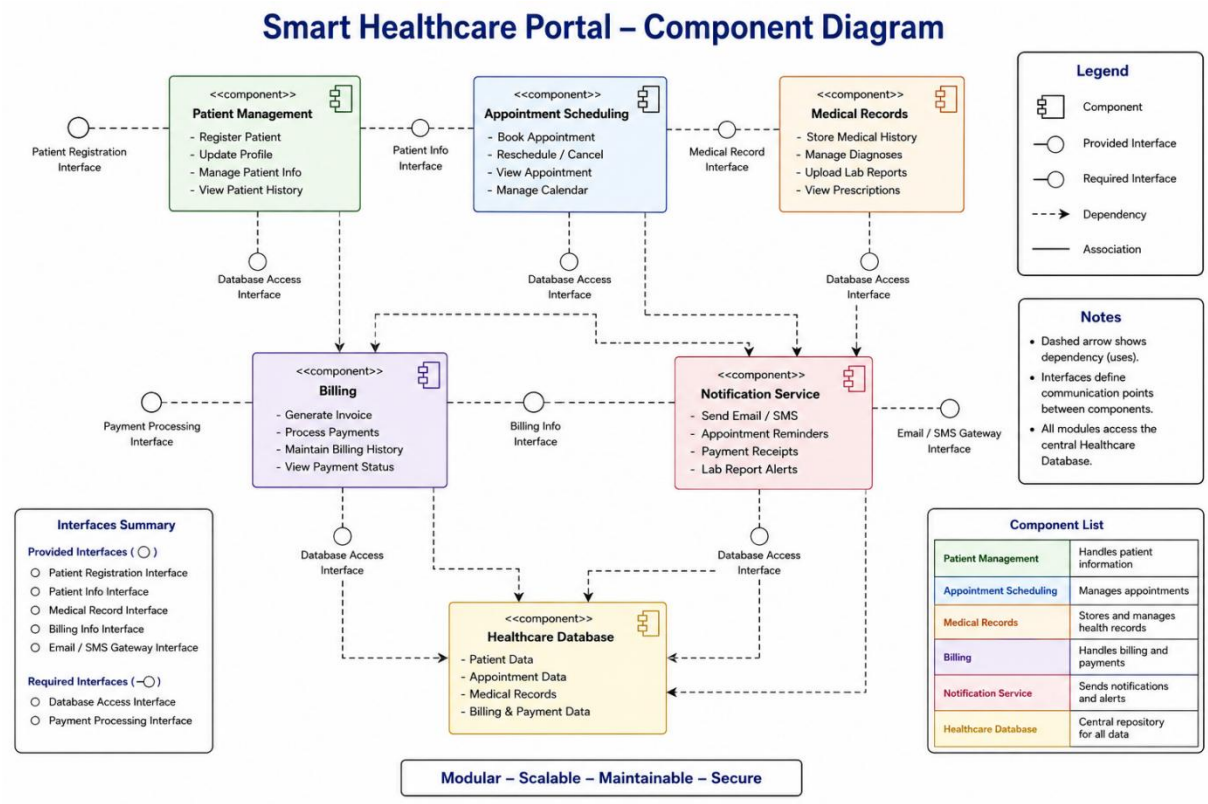
4. Interfaces and Dependencies

Provided Interfaces

- Patient Registration Interface
- Appointment Booking Interface
- Medical Record Interface
- Billing Interface
- Notification Interface

Required Interfaces

- Database Access Interface
- Payment Processing Interface
- Email/SMS Gateway Interface



6. Deployment Diagram Design

A **Smart City Waste Management System** is deployed across IoT waste-bin sensors, edge devices, cloud servers, and monitoring dashboards. The current design lacks clarity regarding physical architecture and communication links.

Analyze the deployment environment, identify nodes, artifacts, and communication links, and construct a detailed Deployment Diagram representing the complete system architecture.

(10 Marks)

(a) Analysis of the Deployment Environment

A **Smart City Waste Management System** uses IoT technology, edge computing, cloud computing, and monitoring dashboards to collect, process, and analyze waste-bin data in real time.

The deployment environment consists of:

- **IoT Waste Bin Sensors** – Detect waste level, temperature, gas leakage, and bin location.
- **Edge Device (Gateway/Raspberry Pi)** – Collects data from nearby sensors, filters unnecessary data, and forwards it to the cloud.
- **Cloud Server** – Stores sensor data, processes analytics, predicts bin fill levels, and manages the database.
- **Database Server** – Stores user information, sensor readings, alerts, and historical records.
- **Web/Application Server** – Hosts APIs and management services.
- **Monitoring Dashboard** – Used by city administrators to monitor bins, alerts, and collection routes.
- **Mobile Application** – Used by waste collection staff to receive optimized routes and notifications.

(b) Nodes

Node	Purpose
IoT Waste Bin Sensor	Measures waste level, gas, temperature, GPS location
Edge Gateway Device	Local processing and communication
Internet/Network	Transfers data securely
Cloud Application Server	Runs waste management services
Database Server	Stores sensor and user data
Monitoring Dashboard	Displays system status

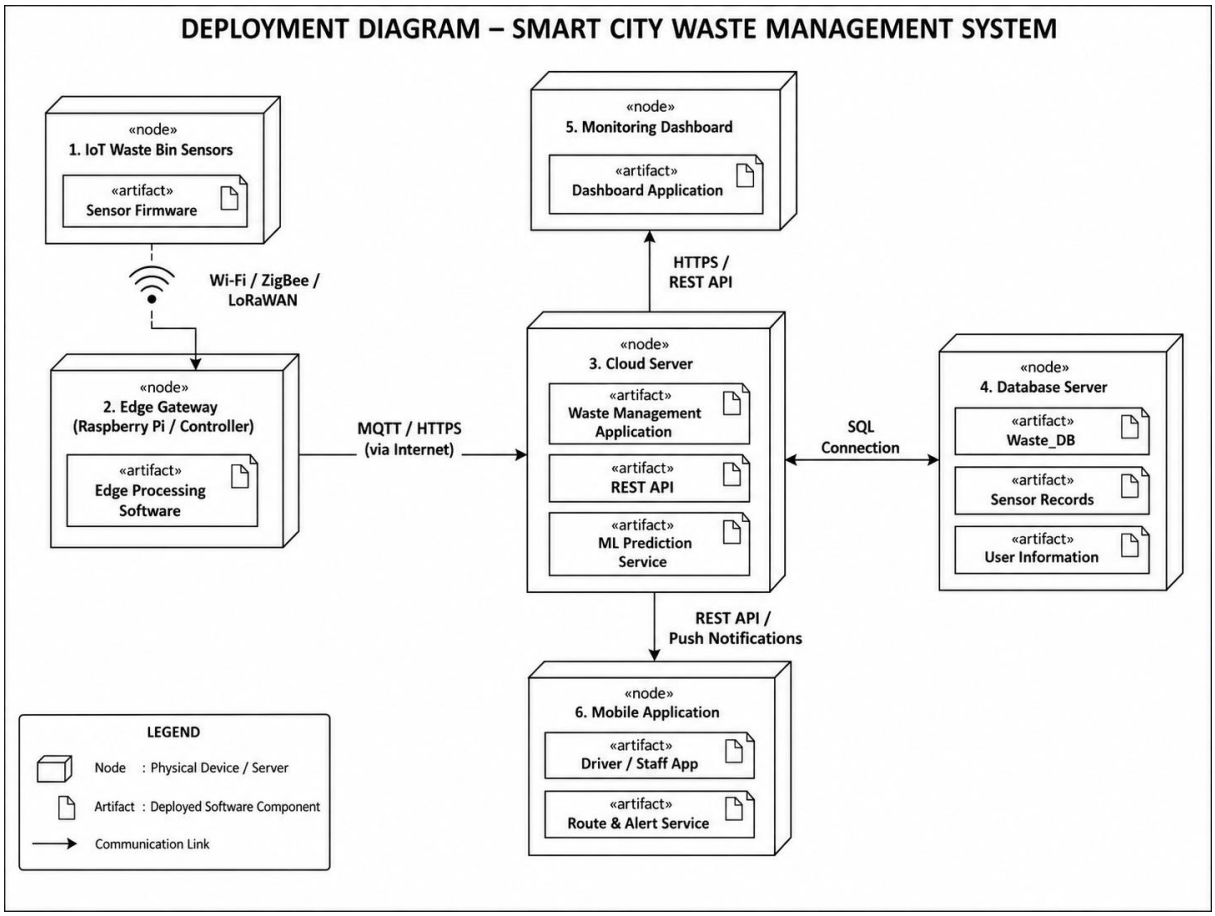
Node	Purpose
Mobile Device	Used by collection staff

(c) Artifacts

- Sensor Firmware
- Edge Processing Software
- REST API Services
- Waste Management Application
- Machine Learning Prediction Model
- Database (Waste_DB)
- Web Dashboard
- Mobile App

(d) Communication Links

- **IoT Sensor → Edge Device**
    - Wi-Fi / ZigBee / LoRaWAN
- **Edge Device → Cloud Server**
    - Internet (HTTPS/MQTT)
- **Cloud Server ↔ Database Server**
    - SQL Connection
- **Cloud Server → Monitoring Dashboard**
    - REST API / HTTPS
- **Cloud Server → Mobile Application**
    - REST API / Push Notification
- **Dashboard ↔ Database**
    - Query Reports


DEPLOYMENT DIAGRAM – SMART CITY WASTE MANAGEMENT SYSTEM
«node»
1. IoT Waste Bin Sensors
«artifact»
Sensor Firmware
Wi-Fi / ZigBee / LoRaWAN
«node»
2. Edge Gateway (Raspberry Pi / Controller)
«artifact»
Edge Processing Software
MQTT / HTTPS (via Internet)
«node»
5. Monitoring Dashboard
«artifact»
Dashboard Application
HTTPS / REST API
«node»
3. Cloud Server
«artifact»
Waste Management Application
«artifact»
REST API
«artifact»
ML Prediction Service
SQL Connection
«node»
4. Database Server
«artifact»
Waste_DB
«artifact»
Sensor Records
«artifact»
User Information
REST API / Push Notifications
«node»
6. Mobile Application
«artifact»
Driver / Staff App
«artifact»
Route & Alert Service
LEGEND
Node : Physical Device / Server
Artifact : Deployed Software Component
Communication Link

RUBRICS FOR CALCULATION

Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand